

Summary Notes: Chapter 5 - Text Clustering and Topic Modeling

Please read this carefully: This will be the only additional part from Quiz-1 to mid sem.

Core Concept:

This chapter explores how to use Large Language Models to find hidden structures and themes in large collections of text without needing labels. This is the heart of **unsupervised learning**. We move from asking a model "what is this?" to asking "what are the hidden patterns in this collection?"

Part 1: The "Why" of Clustering and Topic Modeling

- **Goal:** To automatically organize, summarize, and explore large amounts of text data.
- **Two Key Tasks:**
 1. **Clustering:** Grouping similar documents together (e.g., all customer reviews about "battery life" in one group).
 2. **Topic Modeling:** Discovering the abstract "topics" that run through a collection (e.g., identifying "Technology," "Health," and "Finance" as themes in a set of news articles).

Part 2: The Clustering & Topic Modeling Pipeline

This part breaks down the process into a clear, sequential pipeline. This is the most important part to remember.

Step 1: Text Representation (The "Input")

- We can't feed raw text into a clustering algorithm. We must convert it into numbers (vectors).

Traditional Methods (The Baseline): Bag-of-Words(I hope all of you are familiar

with this term and concept . In ML you have studied about it) or TF-IDF (For those who are new to this term: TF-IDF stands for Term Frequency-Inverse Document Frequency.

It is a numerical statistic used in information retrieval and text mining to reflect how important a word is to a document within a collection (corpus) of documents.

Here is a quick breakdown of the two parts:

- **Term Frequency (TF):** Measures how frequently a term appears in a document. Since different documents have different lengths, this is often normalized (e.g., raw count divided by the total number of words in the document).

- **Inverse Document Frequency (IDF):** Measures how rare or common a term is across the entire corpus. Words that appear frequently in many documents (like "the", "is", "and") have a low IDF score, while words that appear in only a few specific documents have a high IDF score.

Multiplying these two values gives the TF-IDF score. A high TF-IDF score is achieved when a term appears **frequently within a specific document** (high TF) but **rarely across the rest of the corpus** (high IDF) .

- These create sparse vectors based on word counts. They are simple but fail to capture meaning or context (e.g., "car" and "automobile" are treated as completely different).
- **Modern Method (The Power of LLMs): Using Sentence Embeddings.**
 - We feed each document into a pre-trained LLM (like the one powering the `sentence-transformers` library).
 - The model outputs a dense vector (a list of hundreds of numbers) that represents the *meaning* of the entire sentence or document.
 - This is powerful because semantically similar sentences will have similar vectors (they will be close together in a high-dimensional space).

Step 2: Dimensionality Reduction (The "Zoom Out")

- Embeddings have hundreds of dimensions, which is too many for most clustering algorithms to work with effectively (a problem known as the "curse of dimensionality").
- We use techniques to compress the embeddings into fewer dimensions (e.g., 2D or 3D) while preserving the important relationships (distance) between points.
- **Key Technique: UMAP (Uniform Manifold Approximation and Projection)**
 - UMAP is excellent at creating a "map" of the documents, where similar documents are placed close together and dissimilar ones are far apart. It creates tight, well-separated clusters.

Step 3: Clustering (The "Grouping")

- Now we apply an algorithm to find groups in our reduced-dimensionality data.
- **Key Technique: HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise)**
 - Unlike algorithms like K-Means (where you have to specify the number of clusters upfront), HDBSCAN is fantastic for exploratory analysis.
 - It finds clusters of varying densities.
 - It doesn't force every single document into a cluster. It can classify some points as **noise** (outliers that don't fit well into any group). This is a very realistic and useful feature.

Step 4: Topic Representation (The "Interpretation")

- We have groups of documents, but what are they *about*? This is where we interpret the clusters.
- **Method 1: Centroid-based (using TF-IDF):** For each cluster, treat all its documents as a single combined text. Find the words that are most important in this combined text *compared to* the combined text of all other clusters (using TF-IDF). The words with the highest scores are the topic keywords.
- **Method 2: Using an LLM (cTF-IDF):** The book highlights a modern approach called **Class-based TF-IDF (cTF-IDF)** . This scores words based on their importance to a specific cluster versus all other clusters. It often provides more coherent and representative topic descriptions.

At the end the Figures 5-16 and 5-17 tell you the main takeaway from this chapter. Please try to get the concepts.

Now an interesting Assignment:

Assignment on Chapter-5 of our text book: Exploring a News Article Archive

Objective: To apply the unsupervised learning pipeline from Chapter 5 to a real-world text dataset, analyzing the results and interpreting the discovered topics.

Dataset: You will use the "**BBC News Archive**" dataset. This dataset contains 2,225 articles from the BBC news website, originally grouped into 5 categories (business, entertainment, politics, sport, tech). However, for this assignment, you will **ignore the labels** and treat it as an unlabeled collection to see if you can uncover these categories on your own.

(Provide a link to download the dataset, e.g., from a common repository like Kaggle or a direct download link. Or, if you prefer a smaller, quicker dataset, you could use the 20 Newsgroups dataset, which is built into `sklearn`. The first choice is preferable though).

Instructions:

1. **Load and Prepare (10%):**
 - Load the BBC News dataset. The dataset typically has two columns: `category` (the true label, which you will save for later evaluation) and `text` (the raw article content).
 - For this assignment, you will **not** use the `category` column for your model. Keep it aside only for the final analysis.
2. **Generate Embeddings (20%):**

- Use the `sentence-transformers` library to convert the raw article text into embeddings.
 - Choose a pre-trained model. A good starting point is `all-MiniLM-L6-v2` for its speed and decent performance.
 - Your output for this step should be a matrix where each row is an article and each column is a dimension of its embedding vector.
3. **Reduce Dimensionality & Cluster (30%):**
- Apply **UMAP** to reduce the high-dimensional embeddings to a lower number of components. Experiment with `n_components` (e.g., 5, 10, 25) and `n_neighbors`.
 - Apply **HDBSCAN** to the reduced data to find clusters. Pay close attention to the `min_cluster_size` parameter and how it affects the results.
 - **Task:** Print the number of clusters found and the number of articles classified as "noise" by HDBSCAN. How does this compare to the known number of 5 categories?
4. **Interpret the Topics (25%):**
- Implement or use an existing library to perform **cTF-IDF** on the clusters you found.
 - For each cluster, generate a list of the top 5-10 most important words.
 - **Task:** Based on these keywords, give each cluster a human-readable topic label (e.g., "Cluster 0" might have keywords like "goal, match, player, game" -> you would label it "Sports").
5. **Analysis and Reflection (15%):**
- Create a simple visualization (e.g., using Matplotlib) of your UMAP-reduced data, color-coding the points by the clusters HDBSCAN found.
 - Compare your discovered clusters to the original, hidden `category` labels.
 - Did your clusters map well to the original categories?
 - Which articles ended up as "noise"? Look at a few examples. Why do you think HDBSCAN couldn't place them in a cluster? (e.g., were they too short, did they cover multiple topics?)
 - **Write a brief conclusion (1-2 paragraphs)** summarizing what you found. What are the strengths and weaknesses of this unsupervised approach for organizing a news archive?

Deliverables:

1. A Jupyter Notebook (`.ipynb`) containing all your code, visualizations, and markdown cells explaining your steps.
2. A short report (or the final markdown cells in your notebook) answering the questions in Step 5 (Analysis and Reflection).

Hints:

- Use the `hdbscan` and `umap-learn` libraries.

- For cTF-IDF, you can implement it yourself based on the chapter's explanation or use a library like `bertopic`, which incorporates these steps, but try to build the pipeline step-by-step first to understand it.

HAPPY HOLI AND HAPPY LEARNING